

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Master 1° Livello in Cyber Security

Dancing with opcodes

Android Malware Detection via Sonification and Deep Learning

Lecturer:

Dott. Giovanni Ciaramella

Candidate:

Enrico Dolce

ACADEMIC YEAR 2024/2025

Abstract

The sophistication of malware is increasing and becoming more complex as threat actors develop new tactics and techniques to deliver malicious payloads to targeted victims. The implementation of robust detection and containment mechanisms has become crucial to safeguard both organizations and end-user systems.

After an initial analysis of contemporary malware detection methodologies and a discussion of novel techniques, this study proposes a new approach to the identification and classification of malware using static analysis, one of the most widely adopted methodologies. Specifically, this study proposes a new approach by developing a deep learning model for automated malware classification trained on spectrograms derived from Android application opcodes.

The main idea behind the proposed approach is to treat opcodes as a sequence of frequencies, transforming each application into a fingerprint encoded as a spectrogram image so as to preserve the sequential structure of the opcodes and reflect the application logic. Using spectrograms makes the model capture both frequency characteristics and temporal patterns in the opcode stream derived from the applications, offering more features for classification.

This idea has been applied to automatically identify malicious Android Package Files (APK) using a convolutional neural network.

To do so, as a first step, a balanced dataset comprising 5,000 Android applications is built, including trusted and malicious samples (2,500 malicious samples from malware repositories and 2,500 benign applications from trusted sources). Afterward, the dataset is processed into spectrogram images by mapping each opcode to a specific family, subsequently with one of the twelve musical notes at well-defined wave frequencies. Moreover, the experimental analysis was conducted leveraging different deep learning architectures, including InceptionNet and MobileNet to evaluate performance across accuracy, precision, recall, f-measure, AUC and loss.

Once the training and validation phases were concluded, we obtained interesting outcomes during the evaluation. Specifically, we achieved an accuracy of 0.958 and a loss of 0.250 using MobileNet architecture. The results demonstrated the effectiveness of the proposed method for malware classification tasks.

Contents

1 Introduction **5**

1.1 Android Architecture 6

1.2 Malware Application Techniques 7

1.3 Malware Typology 8

1.4 Defensive Adoption and Detection Paradigms 9

1.5 Study Objective 9

2 Related work **11**

3 The method **15**

3.1 Dataset composition 16

3.2 Pre-processing: from opcodes to spectrograms 17

3.3 Experimental setup 21

4 Results **25**

4.1 Training 25

4.2 Metrics and performance 25

4.3 Discussion 30

4.4 Limitations 30

5 Conclusion and future work **33**

5.1 Future work 33

1. Introduction

Malware in the last few decades has been one of the most discussed topics in cybersecurity as they are evolving rapidly and becoming more sophisticated. Although awareness is growing among both end users and organizations, malware is still the most effective and feared cyber threat as documented in a report published by Statista in February 2026 [1].

Threat actors do not only aim to breach companies for money extortion or to steal their sensitive data or intellectual property, starting from the widespread adoption of smartphones, threat actors also target end users and their portable devices. The objectives of these attacks may vary from case to case, the most common motive is to monetize directly by deploying malicious apps and leveraging sms fraud, sell stolen Personally Identifiable Information (PII) or sell credentials. Threat actors may also be interested in conducting espionage using government-sponsored spyware or to massively spread malware to build a botnet infrastructure.

Among mobile platforms, Android stands out as the primary attack surface, holding approximately 70% of the global market share [2] becoming a de facto standard for mobile devices. The first commercial release, Android 1.0, shipped with the HTC Dream (T-Mobile G1) on September 23, 2008 [3]. Since then, Google has maintained an annual release cadence, delivering 16 major versions through 2025, each introducing progressively stronger security mechanisms: from the Dalvik virtual machine in early versions to the Android Runtime (ART), hardware-backed Keystore, runtime permission models (introduced in Android 6.0 Marshmallow), and sandboxed application isolation.

[1] https://www.statista.com/statistics/500946/worldwide-leading-it-security-threats/

[2] https://www.statista.com/statistics/272698/global-market-share-held-by-mobile-operating-systems-since-2009/

[3] https://www.t-mobile.com/news/press/t-mobile-unveils-the-t-mobile-g1-the-first-phone-powered-by

1.1 Android Architecture

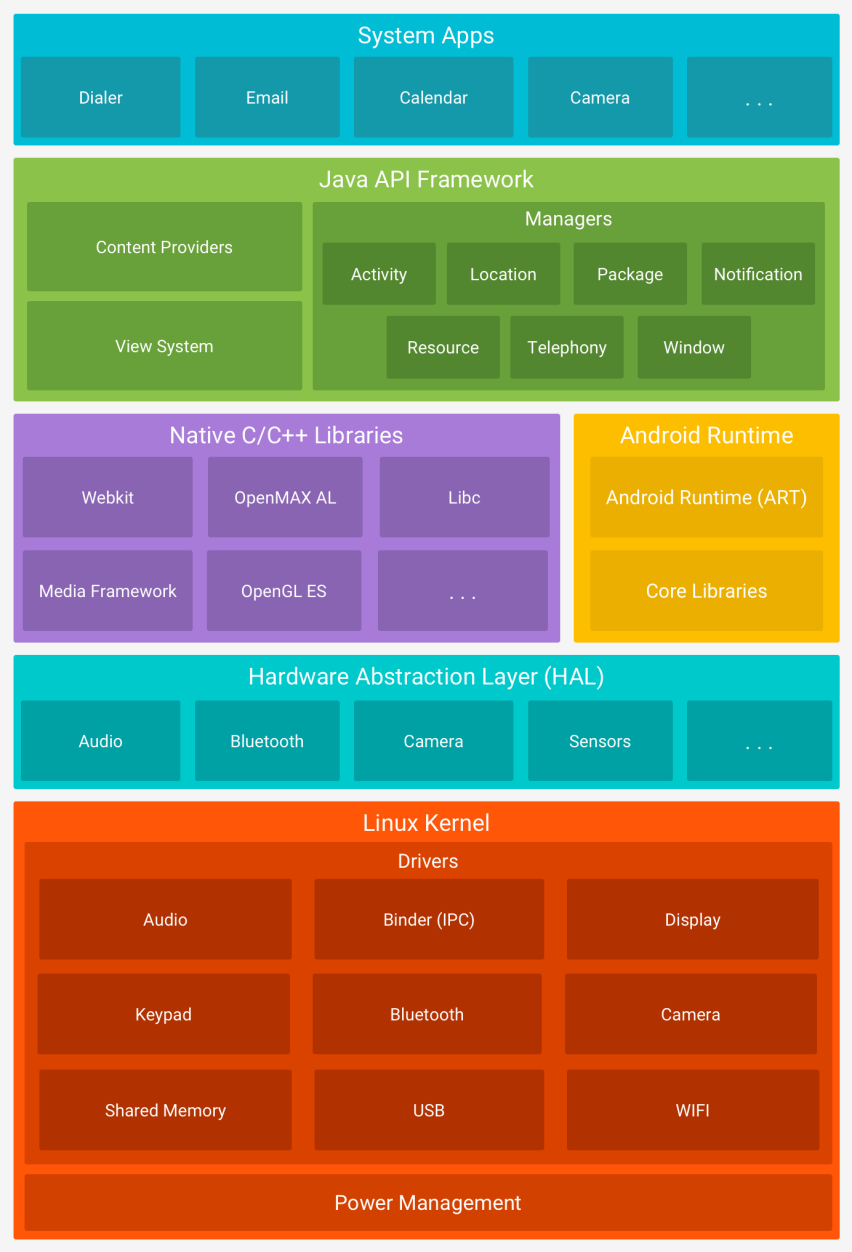


Figure 1.1: Illustration of Android architecture

Android is structured as a layered software stack [4]. At the base, the Linux Kernel provides hardware abstraction, process isolation, and enforces Security-Enhanced Linux (SELinux) mandatory access control policies. Above it, a layer of Native C/C++ Libraries provides fundamental system capabilities consumed by the An-

[4] https://developer.android.com/guide/platform

droid Runtime (ART), which replaced Dalvik starting with Android 5.0 by compiling application bytecode to native machine code via Ahead-Of-Time (AOT) compilation. The API Framework then exposes platform services, such as the Activity Manager, Package Manager, and Location Manager, to the topmost Application Layer where both system and third-party applications execute under the same runtime environment and security policy.

Starting from Android 7.0 (Nougat), Android Runtime (ART) adopted a hybrid execution strategy combining AOT compilation, Just-In-Time (JIT) compilation, and interpretation, allowing the runtime to profile application execution and selectively apply AOT compilation for frequently executed code paths [5]. This evolution significantly improved application startup time and system performance while reducing storage consumption compared to pure AOT approaches.

Android applications are distributed as APK (Android Package) files, which are ZIP-formatted archives containing compiled bytecode in the Dalvik Executable (DEX) format, resources, native libraries, and a manifest file that declares the application's components, permissions, and entry points. Each application component communicates through the inter-process communication mechanism known as Binder, which is the backbone of Android's inter-component communication model.

1.2 Malware Application Techniques

One of the most pervasive techniques employed by Android malware developers is *app repackaging*: attackers decompile a legitimate APK into either its DEX bytecode, the binary instruction format executed by the Android Runtime (ART), or its Smali representation, a human-readable assembly form that enables disassembly, inspection, modification, and reassembly of application logic directly from a ".dex" file. Malicious payloads are then injected and the trojanized application is redistributed through third-party markets [14]. Repackaging effectively defeats signature-based detection since the malware's signature is altered with each injection iteration.

Code obfuscation is systematically used to hinder both static and dynamic analysis. Common obfuscation strategies include identifier renaming, string encryption, control-flow graph manipulation, junk/dead code injection, and reflection-based method invocation.[12]

[5] https://source.android.com/docs/core/runtime/jit-compiler

Dynamic Code Loading (DCL) allows an application to download and execute additional code at runtime, enabling malware to appear benign at install time and install-scan time while loading malicious logic from a remote server after deployment. DCL techniques allow malicious applications to bypass both Google Play's static vetting and dynamic sandbox analysis since the payload is only materialized at runtime.

Android malware also employs anti-analysis techniques such as emulator detection, sandbox detection via user interaction monitoring, and time-based checks to prevent behavioral analysis in automated sandboxes. These techniques are specifically designed to disrupt the feature extraction processes of machine learning-based classifiers, necessitating increasingly sophisticated countermeasures in both static and dynamic analysis pipelines.

1.3 Malware Typology

The Android malware ecosystem encompasses several well-documented threat categories [6]. Trojans are the most prevalent, disguising themselves as legitimate applications to exfiltrate credentials or deliver secondary payloads. Adware represents the highest volume of detections, collecting behavioral data and displaying unsolicited advertisements; according to Kaspersky's 2024 threat report, adware accounts for 57% of attacked users, remaining the most widespread category ahead of general Trojans at 25% [7].

Spyware and Remote Access Trojans (RATs) silently harvest sensitive data including location, contacts, and communications, and are often deployed in targeted espionage campaigns. Banking trojans overlay legitimate financial applications with phishing screens and intercept SMS-based two-factor authentication codes. SMS malware leverages premium-rate messaging services to generate fraudulent charges or intercept OTP codes, while ransomware either locks the device screen or encrypts user files demanding cryptocurrency payment, with a growing focus on enterprise targets [1].

[6] https://www.statista.com/statistics/653688/distribution-of-mobile-malware-type/

[7] https://www.kaspersky.com/about/press-releases/banking-data-theft-attacks-on-smartphones-triple-in-2024-kaspersky-reports

1.4 Defensive Adoption and Detection Paradigms

Defenses against Android malware operate at multiple levels. At the platform level, application sandboxing, Security-Enhanced Linux (SELinux) policies, and runtime permissions collectively enforce the principle of least privilege, limiting the blast radius of a compromised application. At the ecosystem level, Google Play Protect continuously scans applications using machine learning classifiers, though persistent bypass rates for polymorphic and Dynamic Class Loading (DCL) based malware demonstrate that no single layer is sufficient. For end users and organizations, the most impactful measures remain restricting sideloading, maintaining timely OS patch levels, and applying the principle of least privilege when reviewing permission requests.

In response to the threat landscape described above, the research community has converged on three principal detection paradigms. Static analysis inspects decompiled bytecode, the *AndroidManifest.xml* permission declarations, API call sequences, and opcode patterns without executing the application, offering fast, scalable, and execution-safe detection. Dynamic analysis instead monitors runtime behavior, including system calls, network traffic, and resource consumption, trading computational overhead for the ability to observe obfuscated or DCL-based behavior. Finally, hybrid analysis combines both approaches to exploit their complementary strengths, yielding higher detection rates at the cost of increased complexity.

1.5 Study Objective

Starting from these considerations, the aim of this study is to leverage one of the most commonly used malware identification techniques, static analysis, developing a malware identification method based on deep learning by extracting discriminative features such as opcode sequences, offering a deterministic, scalable and execution-safe alternative to dynamic approaches, well suited for large-scale automated pipelines or to run directly on small-end devices such as smartphones.

To conduct this study, we created a dataset of 5,000 Android applications, perfectly balanced into 2,500 malware and 2,500 trusted samples. After that, for each sample, we extracted the opcode sequences and then converted them into images, leveraging a Python script developed by the authors. After the composition of the dataset using images, we trained multiple models using two different architectures:

MobileNet and Inception.

The results show promising performance, with the best model achieving an accuracy of 0.958 and a loss of 0.250, demonstrating that the proposed method can reach engaging performance in Android malware detection.

The study proceeds as follows: Section 2 discusses the current state-of-the-art literature in malware detection, Section 3 describes in detail the proposed spectrogram-based malware detection method, Section 4 discusses and reports the results of the experimental analysis done. Finally, the last section includes the conclusion and outlines possible future research and investigations.

2. Related work

Over the past decade, rapid technological and software advancements have driven extensive research into malware detection, with machine learning emerging as the dominant paradigm in the literature. In response to the escalating growth of mobile threats, researchers have proposed a range of mitigation strategies grounded in machine learning techniques. The proposed method focuses on threat identification by leveraging machine learning and deep learning techniques.

In [4] researchers proposed a method to detect packed malware by exploiting convolutional neural networks. The proposed method performs static analysis and starts from the application binary code to generate an image that represent the input for a set of deep learning classifiers. This approach has shown the ability of the proposed method to detect packed and unpacked malware with interesting performance through MobileNet and DenseNet models, achieving an accuracy of 0.974 and 0.968 respectively. Unlike the raw binary-to-image conversion adopted by [4], the proposed method operates on opcode sequences rather than raw bytecode, producing semantically richer input representations by transforming opcode sequences into frequency representations and subsequently into spectrograms preserving the temporal instruction flow of the application.

The authors in [5] presented DMLDroid, an Android malware detection based on multimodal fusion that leverages three different representations of malware features, including permissions and intents (tabular-based), DEX file representations (image-based), and API calls (graph-derived sequence-based). The approach taken from the authors of a multimodal fusion of three different features shows robust performance achieving an accuracy of 0.980. Although the multimodal fusion strategy of DMLDroid [5] achieves strong performance, it introduces significant complexity. The proposed method deliberately focuses on a single feature (opcode sequences) prioritising simplicity, determinism, and computational efficiency while maintaining competitive detection capability.

In [9], authors proposed a method aimed at detecting the malware belonging family and automatically pointing out a subset of potentially malicious classes by representing an application as an image and classify it with a deep learning model. Then, exploiting the use of activation maps, the approach points out potentially malicious classes obtaining an overall accuracy of 0.944 in the evaluation of a dataset composed of 8,430 real-world Android malware, showing also that the use of activation maps can provide explainability about the deep learning model decision. Similarly to [9], the proposed method extracts opcode sequences from smali code, producing a representation directly tied to the application's executable logic and yielding promising detection results.

Researchers in [11] presented an approach to detect the belonging family of the malicious sample under analysis by representing the executable of the application in terms of audio file and, exploiting audio signal processing techniques, to extract a set of numerical features from each sample to build several machine learning models and evaluate their effectiveness in malware detection. Experiments were conducted on a data-set composed of 50,000 Android real-world samples (24,553 malicious among 71 families and 25,447 legitimate), and reached an accuracy equal to 0.952 in Android malware detection and of 0.922 in family detection. Both approaches exploit audio-domain representations, but [11] relies on hand-crafted features fed into traditional classifiers. The proposed method instead feeds opcode-derived spectrograms directly into a CNN, enabling end-to-end feature learning without manual engineering.

Finally, in [15] the authors introduced MalWave, a novel approach that uses audio signal processing to detect Android malware by converting Dalvik Executable (DEX) file sequences into audio signals. The extracted audio fingerprints are then used as features for classification, addressing malware detection, family classification, and packed malware detection. Evaluated on the AMD and AndroZoo datasets, MalWave achieves an F1-score of 0.826 for malware detection and 0.687 for family classification. MalWave is the closest work in spirit to the proposed method. The key distinction is the input abstraction: MalWave operates on raw DEX byte sequences, while the proposed method uses opcode sequences, a higher-level representation.

The approaches reviewed above demonstrate that both image-based and audio-based representations of Android application code constitute effective feature spaces for deep learning classifiers. However, most works either rely on raw binary representations that lack semantic structure, or require multimodal pipelines that impose significant computational overhead. This study addresses these limitations by com-

bining the semantic precision of opcode sequences with the time-frequency expressiveness of spectrograms, offering a lightweight and end-to-end trainable detection pipeline suitable for large-scale and resource-constrained deployment scenarios.

3. The method

In this section, we present the proposed method for Android malware detection leveraging Convolutional Neural Network (CNN) architectures to train deep learning models on opcode sequences. Figure 3.1 illustrates the workflow pipeline followed to achieve outcomes:

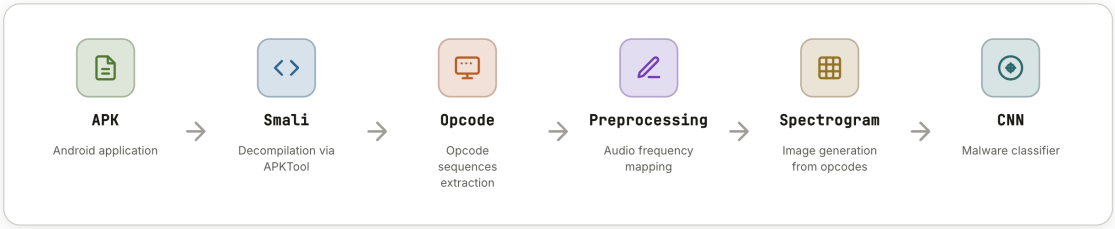


Figure 3.1: Pipeline of the proposed method

- **APK Collection**: As a first step, a set of Android application packages is collected, comprising both benign and malicious samples, with the objective of constructing a balanced and representative dataset for binary classification;
- **Smali Disassembly**: Each APK is subsequently inspected through the APKTool disassembler, which decompiles the application into its corresponding Smali code, exposing the underlying bytecode structure;
- **Opcode Extraction**: From the disassembled Smali code, the opcode sequence of each application is extracted and stored as a plain-text file. This representation preserves the original temporal ordering of instructions, which is essential for capturing behavioral patterns inherent to the execution flow;
- **Preprocessing & Frequency Mapping**: The extracted opcodes are categorized into 12 distinct families based on their functional semantics. Each family is then assigned a unique audio frequency corresponding to a musical note,

establishing a structured mapping between the opcode space and the acoustic domain;

- **Spectrogram Generation**: A custom Python script leverages the aforementioned opcode-to-frequency mapping to convert each application's opcode sequence into a visual spectrogram image. This transformation encodes the temporal distribution of instruction families as a two-dimensional time-frequency representation;

- **CNN-Based Classification**: Once the dataset of spectrogram images is fully assembled, a CNN is trained to perform binary classification, discriminating between benign and malicious applications based on the learned visual patterns within the spectrograms.

3.1 Dataset composition

The composition of the dataset has been built by collecting 5,000 applications and respective opcode sequences. In particular, the dataset that has been used for this study is composed of 2,500 trusted applications and 2,500 malware applications, resulting in a perfectly balanced dataset. Each application was represented as a sequential list of opcode instructions extracted through analysis of the corresponding apk application via Apktool [1]. The trusted applications were sourced from well-known legitimate software repositories, while the malware samples were gathered from publicly available malware databases, ensuring a diverse coverage of malware families and behaviors.

The dataset was subsequently partitioned into training, testing and validation with proportions of 80%, 10%, and 10% respectively, corresponding to 4.000 samples for training, 500 for testing, and 500 for validation, out of a total of 5.000 samples. It is worth noting that, despite the balanced overall composition, the dataset encompasses a heterogeneous variety of malware types, reflecting realistic threat diversity encountered in operational environments.

[1] https://github.com/iBotPeaches/Apktool

3.2 Pre-processing: from opcodes to spectrograms

A crucial step of the training pipeline is the pre-processing phase in which each application represented by its opcode sequence stored in a plain-text file is elaborated to obtain a spectrogram image.

To do so, twelve notes with respective frequencies have been defined.

```
NOTES = {
    "C4": 261.63,
    "C#4": 277.18,
    "D4": 293.66,
    "D#4": 311.13,
    "E4": 329.63,
    "F4": 349.23,
    "F#4": 369.99,
    "G4": 392.00,
    "G#4": 415.30,
    "A4": 440.00,
    "A#4": 466.16,
    "B4": 493.88,
}
```

Knowing that opcodes instructions are more than twelve, we have classified each opcode instruction to respective families of instructions, so that afterwards we could map a specific note to the specified family instead of mapping it to each opcode instruction.

```
CATEGORY_TO_NOTE = {
    # Baseline/control flow - Lower frequencies
    'nop': 'C4',
    'return': 'C4',
    'goto': 'C#4',
    'switch': 'C#4',
    'if': 'D4',
    'compare': 'D4',

    # Mathematical operations - Lower-mid frequencies
    'unary': 'E4',
    'arithmetic': 'F4',
    'arithmetic_2addr': 'F4',
    'bitwise': 'F#4',
    'bitwise_2addr': 'F#4',
    'literal_16': 'G4',
    'literal_8': 'G4',

    # Data movement - Mid-low frequencies
    'move': 'D#4',
    'const': 'E4',

    # Type operations - Mid frequencies
    'typecheck': 'F4',
    'conversion': 'F4',
```

```
    # Array operations - Mid frequencies
    'array_length': 'F#4',
    'array_data': 'F#4',
    'aget': 'G4',
    'aput': 'G4',

    # Field operations - Mid-high frequencies
    'iget': 'G#4',
    'iput': 'G#4',
    'sget': 'A4',
    'sput': 'A4',

    # Critical operations - Higher frequencies
    'new': 'A#4',
    'invoke': 'B4',

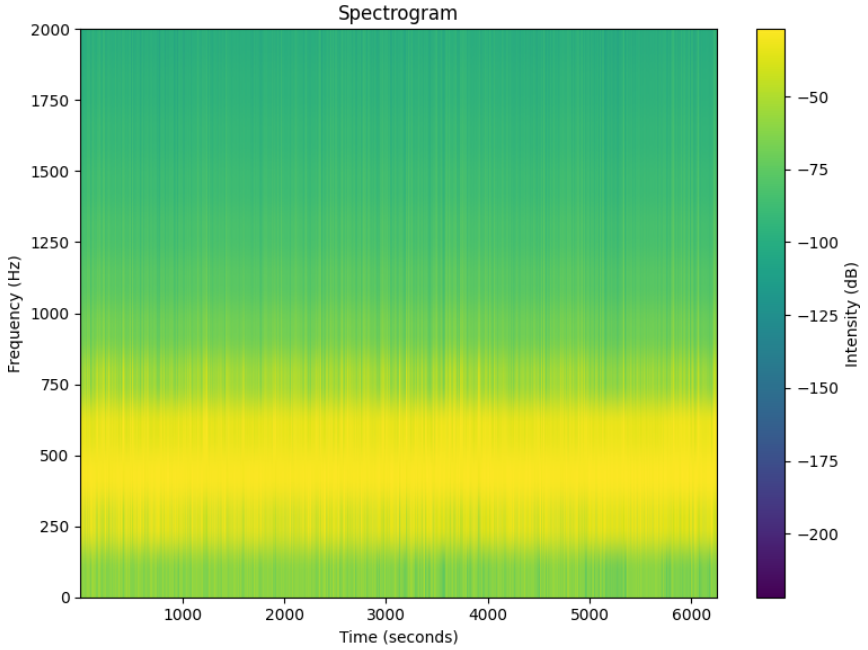
    # Exception/synchronization - High frequency
    'throw': 'A#4',
    'monitor': 'A4',
}
```

Once the frequency-to-opcode mapping has been done, the applications represented by the sequence of opcodes can be easily converted into a spectrogram by a python script. Key consideration to be done in this context concerns the inherent disparity in length and complexity between benign and malicious applications, trusted software tends to exhibit significantly longer opcode sequences compared to malware samples, which are often purpose-built and compact. To mitigate the risk of introducing a length-based bias into the classification model, a fixed sequence length with a randomized variance was defined, allowing padding and truncation to be applied uniformly across all samples. Padding was specifically performed by filling the frequency gaps introduced by the defined fixed-length opcode sequences through the addition of a series of 'D4' notes, deliberately chosen as a neutral note that would not introduce additional spikes into the spectrogram, nor any bias into the model.

Below are presented some examples of normalization with a fixed sequence length of 2,500 opcodes and illustration of the spectrogram representations before and after the normalization strategy has been applied.

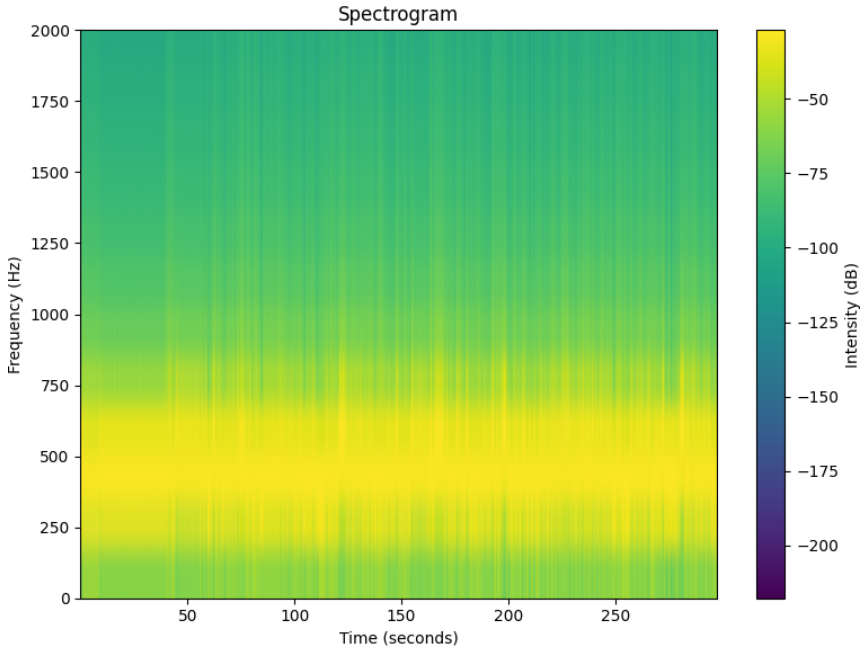
Sample	**No normalization**	**Normalization applied**
Trusted	232,553 opcode	2,515 opcode
Malware	2,899 opcode	2,522 opcode

Table 3.1: Average opcode number per each sample type



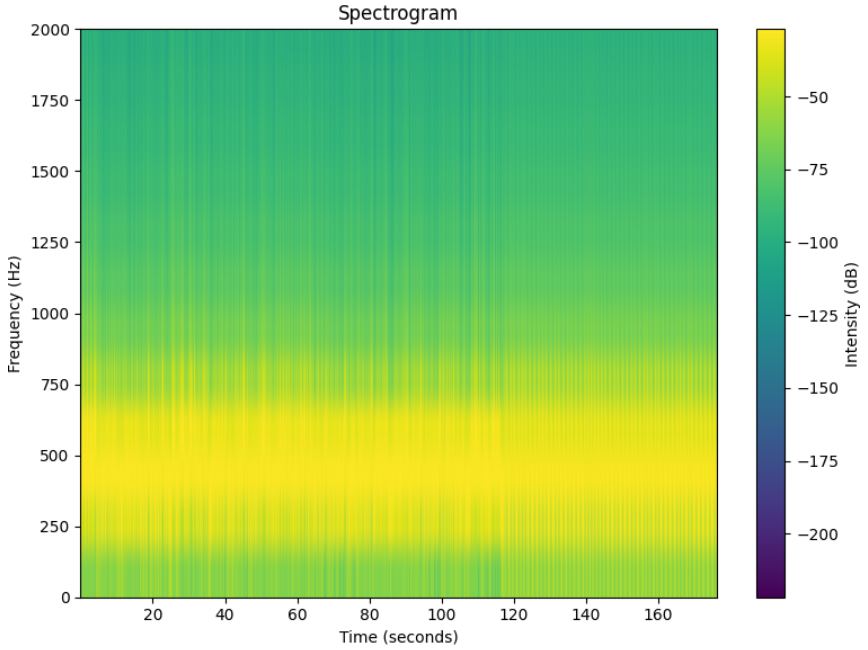
SHA-256: 00000989f3e215ba9fc3bdd5b56af751343b540c1026bf42aea8febf68874ecc

Figure 3.2: Trusted apk spectrogram without normalization



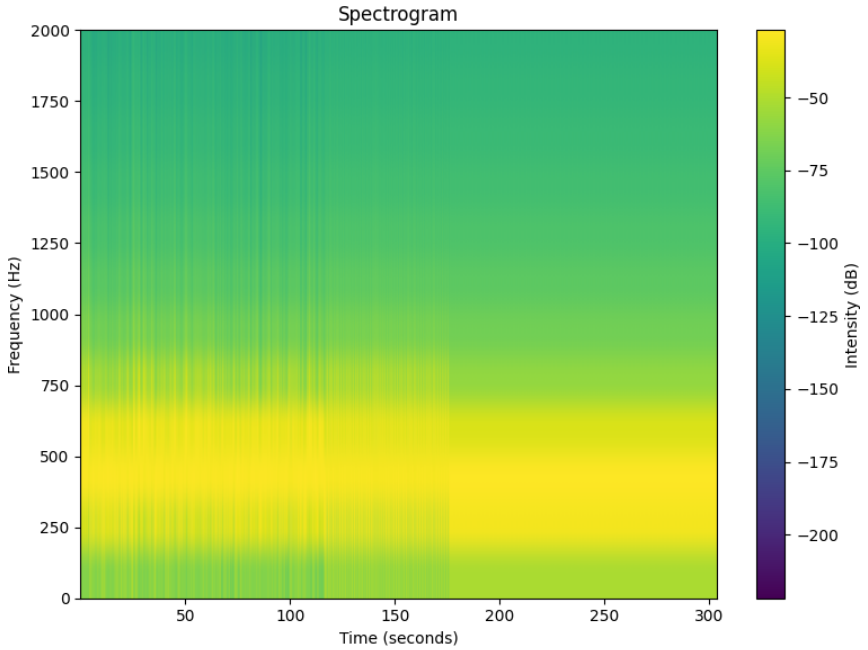
SHA-256: 00000989f3e215ba9fc3bdd5b56af751343b540c1026bf42aea8febf68874ecc

Figure 3.3: Trusted apk with normalization applied



SHA-256: 0002f45e396b4d948ed0e9d2f818ec9f1b10f592347d43825ca2ff6a271a6867

Figure 3.4: Malware apk without normalization



SHA-256: 0002f45e396b4d948ed0e9d2f818ec9f1b10f592347d43825ca2ff6a271a6867

Figure 3.5: Malware apk with normalization applied

The illustrations above highlight the key differences between normalized and non-normalized samples. Trusted applications that have not undergone normalization tend to produce excessively long opcode sequences, resulting in spectrograms that

are overly dense and compressed, a condition that causes the loss of discriminative features critical to the training phase. This issue is addressed through the truncation procedure described before.

On the other hand, a subset of malware samples exhibits the opposite characteristic, their opcode sequences are too short to generate meaningful spectrograms. In such cases, the padding strategy introduced earlier extends the spectrogram to the required length by appending a predefined neutral note, preserving structural consistency across the dataset without introducing misleading artifacts.

This normalization strategy ensures that the resulting spectrograms maintain consistent dimensionality and density, preventing the model from relying on opcode sequence length or spectrogram visual density as discriminative features rather than on the actual behavioral patterns encoded within the instruction sequences.

3.3 Experimental setup

To train the deep-learning classifier, two widely adopted CNN architectures were selected: MobileNet and Inception with an input image size of 100x3, representing a height and width of 100 pixels and 3 color channels.

MobileNet, introduced by Howard et al. at Google in 2017 [8], was designed from the ground up to operate efficiently under computational constraints. Its design relies on a streamlined architecture that employs depthwise separable convolutions to build light weight deep neural networks. This makes it a compelling choice in scenarios where training resources are limited and rapid inference is desirable, conditions that closely match the experimental context of this work.

The Inception architecture, proposed by Szegedy et al. and presented at CVPR 2015 [13], incorporates factorization strategies and architectural refinements that significantly reduce computational costs while improving accuracy. This improved efficiency is achieved through a carefully crafted design that enables the network to grow in both depth and width without increasing the overall computational budget. This approach proved highly effective, establishing it as a benchmark reference in the deep learning community. Its strength lies in learning diverse and complementary features, which can be particularly advantageous when the input data, such as opcode spectrograms, carries information distributed across different levels of granularity.

Both architectures were initialized with ImageNet pre-trained weights and fine-

tuned on the opcode spectrogram dataset. Fine-tuning rather than full retraining was adopted to mitigate overfitting, given the limited computational resources and limited size of available labeled malware samples. The key hyperparameters explored across training runs were the learning rate, number of epochs, and batch size, as summarized in Table 3.2 below. Rows in bold highlight the best-performing configuration for each architecture.

Model	Image size	Learning rate	Epochs	Batch size
Inception	100x3	0.0005	30	32
Inception	**100x3**	**0.0001**	**30**	**32**
MobileNet	100x3	0.001	50	32
MobileNet	100x3	0.0005	47	32
MobileNet	**100x3**	**0.0001**	**35**	**32**

Table 3.2: Experimental CNN training hyper parameters

The experimental flow followed a fine tuning approach. Training began with a standard learning rate of 0.001, then progressively reduced to 0.0005 and 0.0001 to observe convergence behavior. A batch size of 32 was held constant across all runs as a controlled variable, isolating the effect of learning rate on model convergence. The objective of this Table 3.2 was to identify the hyperparameter configuration that maximizes classification accuracy while minimizing overfitting.

Too high learning rate does not let the model converge properly both for MobileNet and Inception architecture as shown in Figure 3.7 and Figure 3.6

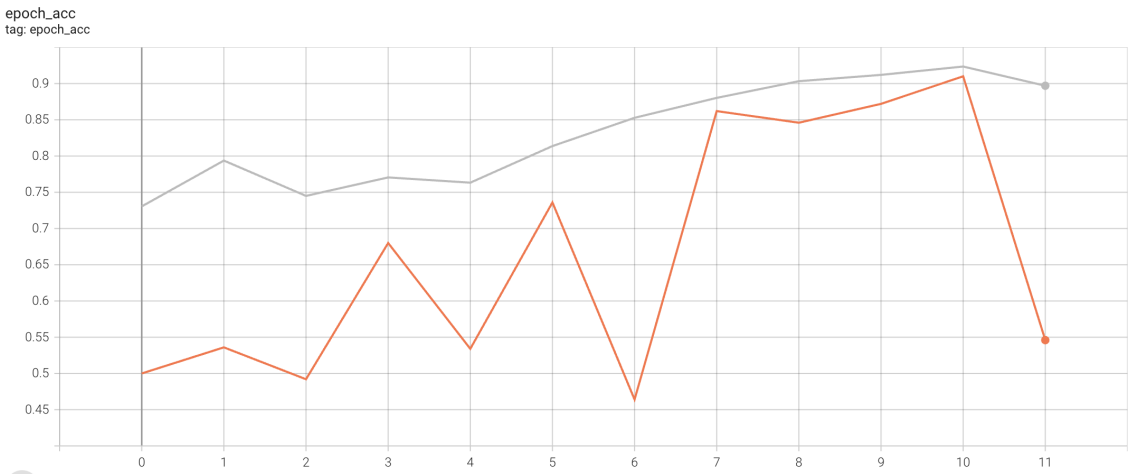


Figure 3.6: Inception instability on learning rate 0.0005 (stopped earlier)

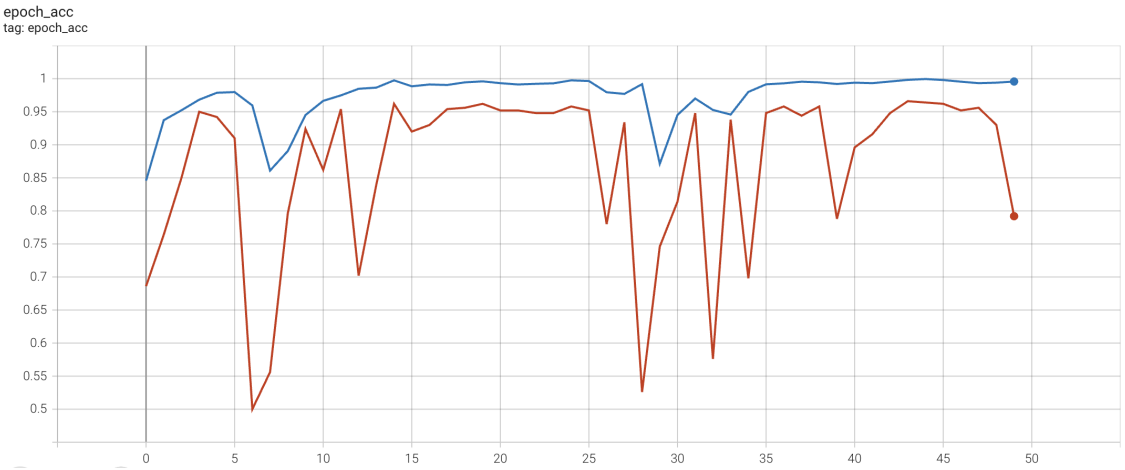


Figure 3.7: MobileNet instability on learning rate 0.001

4. Results

In this chapter we present the outcomes of the training and evaluation phase for both CNN architectures examined in this study. We first describe the training approach used during the training of the CNN model, we then illustrate metrics and chart discussing results obtained.

4.1 Training

Both MobileNet and Inception model architecture were trained following the best hyper parameter identified in Table 3.2. As mentioned before the dataset was partitioned into training, testing and validation with proportions of 80%, 10%, and 10% respectively, resulting in 4,000 training samples, 500 validation samples and 500 test samples.

Training was monitored through three key metrics, via accuracy, loss, and AUC, tracked on both the training and validation splits at each epoch.

4.2 Metrics and performance

Table 4.1 summarizes the classification performance of both architectures evaluated on the held-out test set across six metrics: loss, accuracy, precision, recall, F-measure, and AUC.

Model	**Loss**	**Accuracy**	**Precision**	**Recall**	**F-Measure**	**AUC**
Inception	0.377	0.948	0.948	0.948	0.948	0.955
MobileNet	0.250	0.958	0.958	0.958	0.958	0.983

Table 4.1: Model performance metrics obtained during CNN training tests

MobileNet achieved the best overall performance, recording an accuracy of 0.958

and an AUC of 0.983, with a notably lower loss of 0.250 compared to Inception. The Inception architecture, while competitive, yielded a slightly lower accuracy of 0.948 and an AUC of 0.955, with a higher loss of 0.377. It is worth noting that, for both models, precision, recall, and F-measure converge to the same value as accuracy, a characteristic consistent with a balanced binary classification dataset and symmetric error distribution across the two classes.

Below we illustrate the training performance of both MobileNet and Inception architecture based on accuracy, loss and AUC metrics.

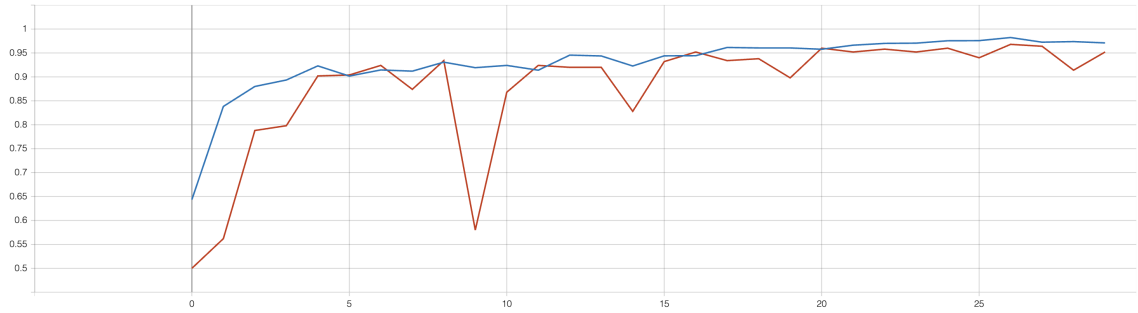


Figure 4.1: Inception accuracy training vs validation

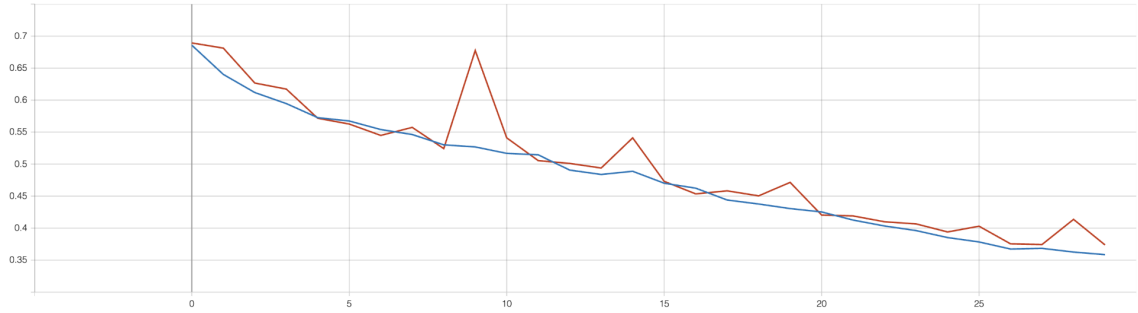


Figure 4.2: Inception loss training vs validation

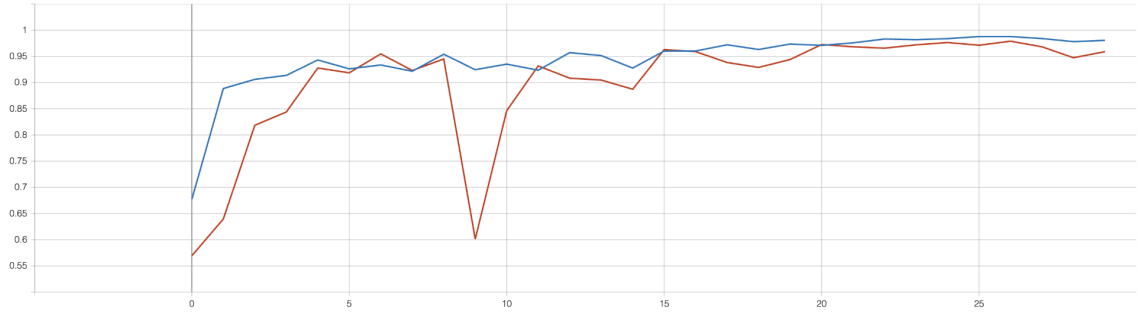


Figure 4.3: Inception AUC training vs validation

The Inception training curves (Figures 4.1–4.3) reveal three distinct phases. Dur-

ing the initial epochs (0–5), all metrics improve rapidly, with training accuracy and AUC rising from approximately 0.65 to 0.90, accompanied by a steep reduction in loss. A transient instability emerges around the 9th epoch, manifesting as a simultaneous spike in validation loss and a corresponding drop in both validation accuracy and AUC to approximately 0.60. Following recovery, the model stabilises through the final epochs, with training and validation curves converging closely around an accuracy and AUC of 0.95–0.97 and a loss of approximately 0.37, indicating the absence of significant overfitting. Nevertheless, the mid-training disruption highlights Inception's susceptibility to hyperparameter selection under constrained dataset conditions.

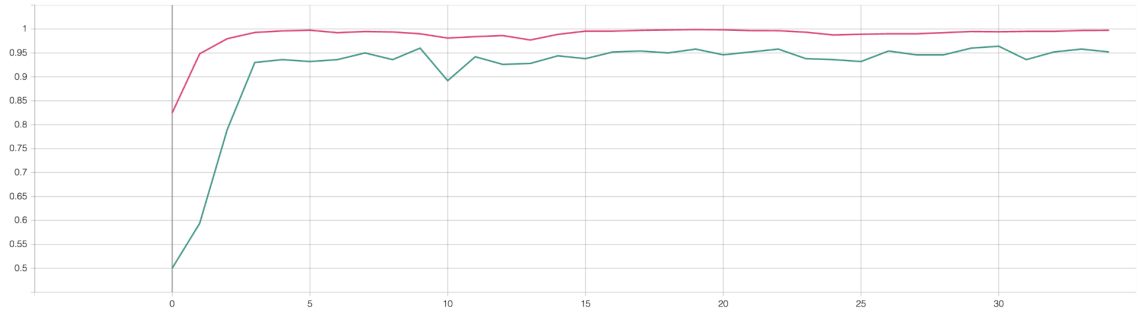


Figure 4.4: MobileNet accuracy training vs validation

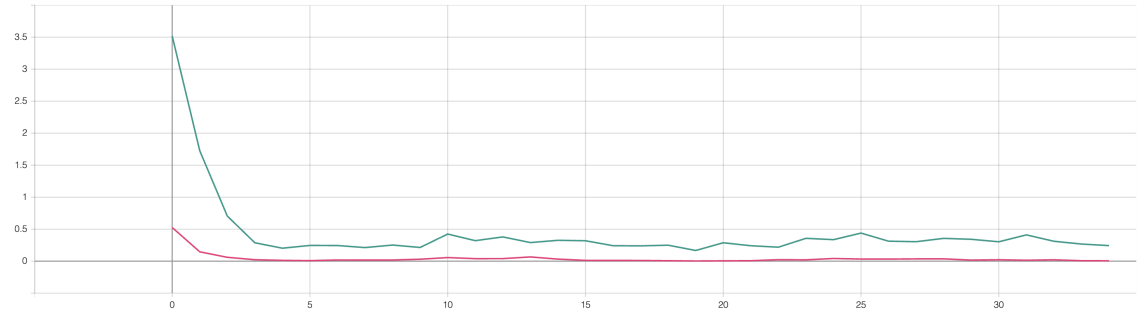


Figure 4.5: MobileNet loss training vs validation

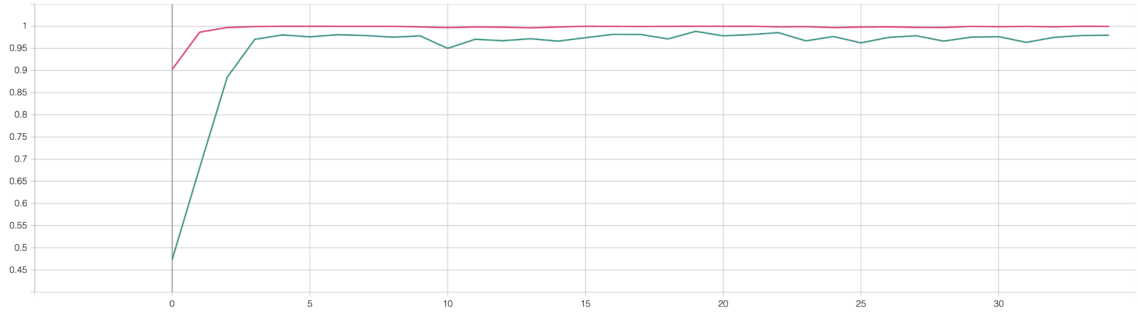


Figure 4.6: MobileNet AUC training vs validation

The MobileNet training curves (Figures 4.4–4.6) exhibit markedly more stable dynamics compared to Inception. From the first epoch, the validation curves immediately plateau at near-optimal values, a behaviour attributable to the strong inductive bias provided by ImageNet pre-trained weights, which required minimal adaptation to the spectrogram domain. A minor transient dip in training accuracy is observable around the 9th epoch, yet it remains localised and does not propagate to validation metrics, confirming the model's robustness under the adopted learning rate. Beyond the 10th epoch, all three metrics stabilise with no appreciable divergence between training and validation splits through the final epoch, indicating well-controlled generalisation and the absence of overfitting. This consistent convergence behaviour, combined with the final test metrics of accuracy 0.958 and AUC 0.983, establishes MobileNet as the optimal architecture for the proposed sonification-based malware detection pipeline under the given resource and dataset constraints.

The confusion matrices obtained from both the Inception and MobileNet deep learning architectures are presented below, providing a comparative evaluation of their respective classification performance.

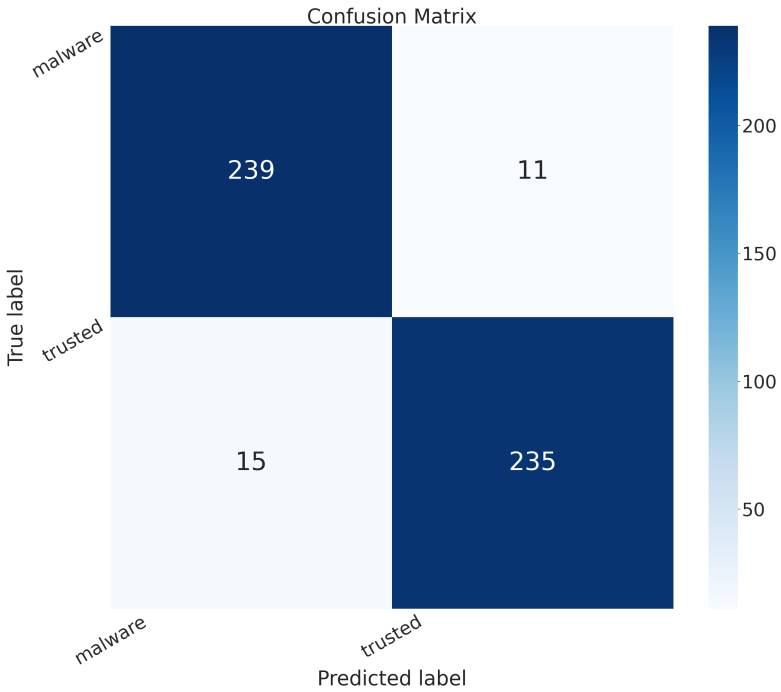


Figure 4.7: Confusion matrix of Inception model

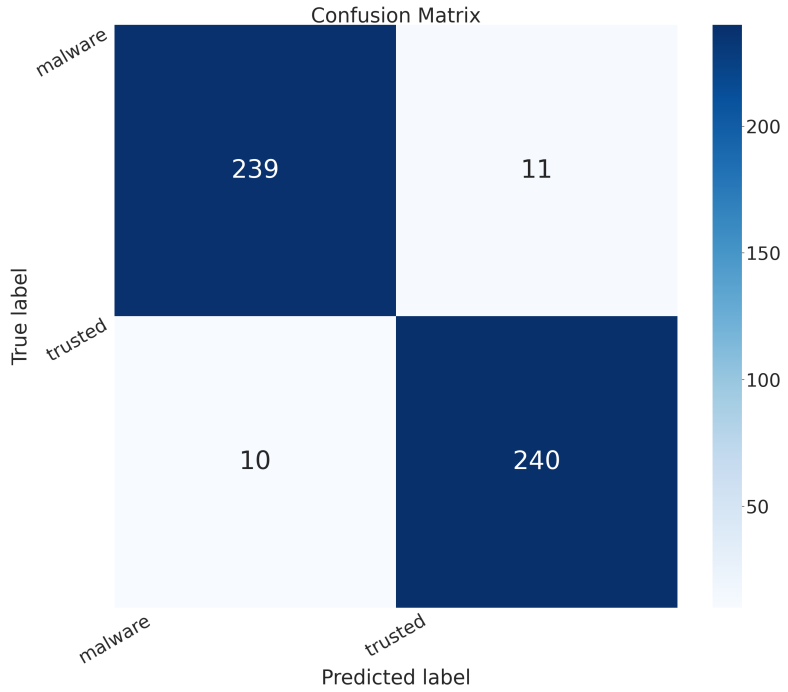


Figure 4.8: Confusion matrix of MobileNet model

To assess the generalization capability of the two proposed architectures, both models were evaluated on a held-out test set comprising 500 samples, equally distributed between the two classes (250 malware and 250 trusted). The results are summarized through the confusion matrices reported in Figures 4.7 and 4.8, where rows represent the true class labels and columns represent the predicted class labels. Cell values denote raw sample counts, and the color intensity is proportional to the frequency of predictions in each cell.

MobileNet achieved an overall accuracy of 0.958, correctly classifying 239 out of 250 malware samples and 240 out of 250 trusted samples. The model produced 11 false negatives, malware instances incorrectly predicted as trusted, and 10 false positives, corresponding to trusted software misclassified as malware. These results confirm a well-balanced classification performance across both categories.

Inception achieved an overall accuracy of 0.948, with 239 malware samples correctly identified and 235 trusted samples. The model produced 11 false negatives and 15 false positives. While both models exhibit comparable recall on the malware class, MobileNet demonstrates a higher precision on the malware class, indicating fewer erroneous classifications on benign software.

Both architectures produce an identical number of false negatives (11), suggesting

equivalent sensitivity to malicious patterns. However, MobileNet's lower false positive rate makes it preferable in deployment scenarios where minimizing disruption to trusted processes is a priority. Overall, the confusion matrices confirm that both models generalize effectively to unseen data, with MobileNet showing a modest but consistent advantage in classification precision.

4.3 Discussion

Both architectures demonstrated strong and consistent classification capability on the sonified opcode spectrogram dataset, confirming the viability of the proposed detection pipeline. MobileNet outperformed Inception across all reported metrics, achieving improvement in accuracy and AUC, while simultaneously exhibiting a lower and more stable loss throughout training, as illustrated in Figures 4.4–4.6.

Inception showed higher sensitivity to learning rate selection, instability was observed at a learning rate of 0.0005 (Figure 3.6) requiring earlier stopping and limiting convergence. This suggests that Inception demands more careful hyperparameter tuning and potentially a larger dataset to fully exploit its architectural depth.

Overall, the results indicate that opcode-derived spectrogram images constitute an effective and semantically meaningful feature space for CNN-based malware classification. The alignment between training and validation curves across both architectures (Figures 4.1–4.6) confirms that the adopted fine-tuning strategy successfully mitigated overfitting, despite the moderate dataset size of 5,000 samples. These findings position the proposed method as a computationally efficient and practically deployable solution for Android malware detection in resource-constrained environments.

4.4 Limitations

Despite the encouraging results, several limitations must be acknowledged. The malware landscape evolves continuously, new obfuscation strategies, packing techniques, and attack families emerge regularly, meaning that a model trained on a static dataset is inherently time-bound, it will degrade in detection capability as novel malware variants appear. To address this problem a periodic retraining is essential to embed those new techniques and malware families that previous model may be weak or fail to detect.

This is, however, difficult to achieve in practice as collecting and correctly labeling new malware samples is a time-consuming process that requires access to up-to-date threat intelligence feeds and controlled analysis environments. In addition, static analysis methods are vulnerable to evasion techniques, attackers nowadays can easily produce multiple variants and transformations of the same malware that exhibit identical runtime behavior yet differ substantially in their opcode sequence.

A further limitation concerns the preprocessing of opcode sequences prior to spectrogram generation. The opcode sequences extracted from trusted applications were observed to be substantially longer than those extracted from malware samples, a disparity attributable to the typically larger codebase and richer functionality of legitimate applications. To enforce uniform input sequences, opcodes that surpassed the fixed-length limit were truncated. This truncation introduces a potential loss of discriminative information located in the tail regions of longer sequences, and may affect the representation of benign samples. Alternative length-normalization strategies should be investigated, including approaches like sliding-window based segmentation or padding sequences with ad-hoc embeddings, to preserve the full opcode distribution without imposing a hard truncation boundary.

The above limitations underscores why the current industry consensus favors combining static and dynamic analysis into hybrid detection pipelines, where behavioral signals at runtime complement the structural patterns captured statically.

5. Conclusion and future work

This study investigated the viability of using Convolutional Neural Network classifiers trained on spectrogram representations derived from opcode sequences as a means of statically identifying Android malware. The results obtained during this study demonstrate that the proposed approach of training a CNN classifier model upon a equally balanced dataset of spectrograms derived from trusted and malware Android applications can be pursued to achieve promising classification performance, even under the constraint of limited resources. The method is particularly well suited for deployment on resource-constrained environments such as mobile devices, legacy systems and IoT endpoints, contexts where traditional dynamic analysis pipelines are often impractical.

In conclusion, this work demonstrates that sonification of opcode sequences offers a practical and computationally efficient input representation for CNN-based malware classification, while also outlining a clear roadmap of open challenges that future research can address to advance the method toward practical, real-world adoption.

5.1 Future work

This study was conducted on a single, curated list of Android applications. Although the resulting findings are promising, it must be highlighted that we cannot make definitive claims about the generalizability of the proposed method to other operating systems, executable formats, or malware families without conducting additional, broader experiments.

This inherent limit does not diminish the importance of the results, rather, it delineates a clear set of open questions and possibilities for improvements in subsequent investigations. Follow-up work can be built on this study along two main directions.

First, evaluating the model on dataset that include obfuscated or packed samples would provide a strong assessment of how robust the spectrogram representation is, revealing its sensitivity boundaries to code transformation techniques and providing a clearer understanding of the conditions under which a sonification-based approach may experience performance degradation. Second, applying the same sonification approach to Windows PE executables or Linux ELF binaries would represent a validation step to determine whether the method extends beyond the Android ecosystem, potentially establishing it as a platform-independent modality for malware analysis.

Bibliography

[1] Abdulrahman Alzahrani et al. Characterization of Android malwares and their families. *ACM Transactions on Software Engineering and Methodology*, 2025. Accessed: April 2026.

[2] G. Ciaramella, F. Martinelli, C. Peluso, A. Santone, and F. Mercaldo. A method for real-world privacy-preserving Android malware detection through federated machine learning. *Information and Software Technology*, page 107892, 2025.

[3] G. Ciaramella, F. Martinelli, A. Santone, and F. Mercaldo. Explainable malware detection by means of federated machine learning. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*, 2025.

[4] G. Ciaramella, F. Martinelli, A. Santone, and F. Mercaldo. A method for packed (and unpacked) malware detection by means of convolutional neural networks. In *Proceedings of the 22nd International Conference on Security and Cryptography (SECRYPT)*, 2025.

[5] Minh Trung Doan and The Duy Phan. DMLDroid: Deep multimodal fusion framework for Android malware detection with resilience to code obfuscation and adversarial perturbations, 2025.

[6] Federico Gerardi, Giacomo Iadarola, Fabio Martinelli, Antonella Santone, and Francesco Mercaldo. Perturbation of image-based malware detection with smali level morphing techniques. In *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/-SocialCom/SustainCom)*, pages 1651–1656. IEEE, 2021.

[7] Nor Zakiah Gorment, Ali Selamat, Lim Kok Cheng, and Ondrej Krejcar. Machine learning algorithm for malware detection: Taxonomy, current challenges, and future directions. *IEEE Access*, 2023.

[8] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications, 2017.

[9] Giacomo Iadarola, Rosangela Casolare, Fabio Martinelli, Francesco Mercaldo, Christian Peluso, and Antonella Santone. A semi-automated explainability-driven approach for malware analysis through deep learning. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021.

[10] Giacomo Iadarola, Fabio Martinelli, Francesco Mercaldo, and Antonella Santone. Towards an interpretable deep learning model for mobile malware detection and family identification. *Computers & Security*, page 102198, 2021.

[11] Francesco Mercaldo and Antonella Santone. Audio signal processing for Android malware detection and family classification. *Journal of Computer Virology and Hacking Techniques*, 17:1–14, 2021.

[12] Janaka Senanayake, Harsha Kalutarage, and Mhd Omar Al-Kadri. The rise of obfuscated Android malware and impacts on detection methods. *PeerJ Computer Science*, 8:e907, 2022.

[13] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. arXiv preprint arXiv:1409.4842.

[14] Kun Tian, Danfeng Yao, Barbara G. Ryder, and Gang Tan. Detection of repackaged Android malware with code-heterogeneity features. *IEEE Transactions on Dependable and Secure Computing*, 17(6):1288–1300, 2020.

[15] Efstratios Vasilellis, Thanos Katsiolis, et al. The sound of malware: an audio fingerprinting malware detection method. *International Journal of Information Security*, 24(3), June 2025.